CU InSpace

Radio Packet Format

2024-02-27

Matteo Golin
Samuel Dewan

Contents

1 Introduction **1**

2 Packet Layout **2**
2.1 Packet Header 2
2.2 Block Header 4

3 Block Formats **6**
3.1 Data Blocks 6
3.1.1 Debug Message 6
3.1.2 Altitude 7
3.1.3 Temperature 7
3.1.4 Pressure 8
3.1.5 Acceleration 8
3.1.6 Angular Velocity 9
3.1.7 GNSS Location 9
3.1.8 GNSS Metadata 11
3.1.9 Humidity 12

1 Introduction

In order to efficiently communicate telemetry data between various devices, Carleton University In Space has devised its own unique packet format for transmission over LoRa radio. The packet format is not only used for radio communications, but also for logging telemetry data in-flight.

This packet format is made publicly available via the GNUGPLv3 license.

The packet format which follows is used for three primary purposes:

- Transmitting telemetry data.
- Controlling telemetry systems.
- Transmitting devices statuses.

2 Packet Layout

CU InSpace's radio packets consist of a packet header followed by one or more segments called blocks. Each block has its own header followed by a variable amount of data. All headers and data segments have a length that is a multiple of four bytes. The layout of a packet is shown in figure 2.1.

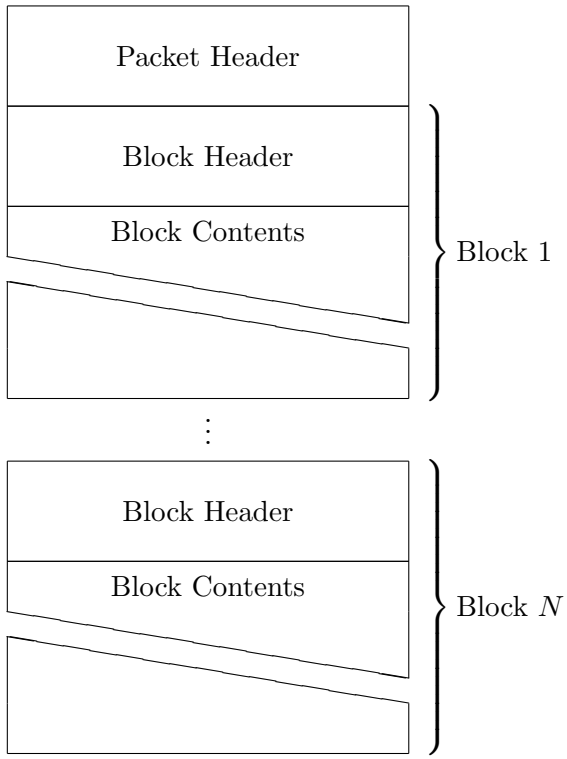

Figure 2.1: Layout of radio packet

2.1 Packet Header

Every packet is preceded by a 16 byte header. The header contains an amateur radio call sign and the packets total length as well as other information which describes the packet as a whole. The packet header follows the format described in figure 2.2.

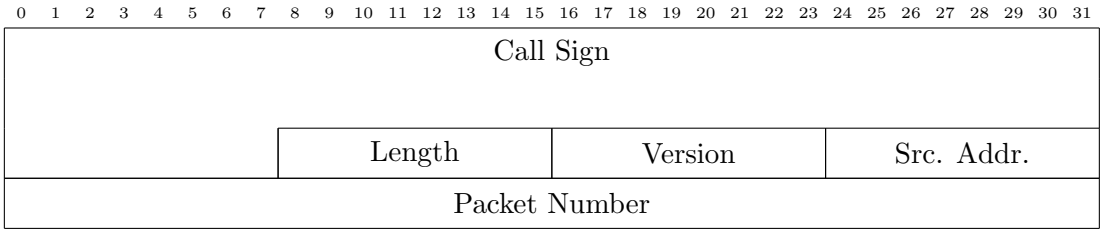

Figure 2.2: Packet header format

Call Sign The first 9 bytes of the packet must contain an amateur radio call sign in ASCII. If the call sign being used is less than 9 bytes long, the call sign will be padded with NULL characters at the end.

The call sign field is 9 bytes long because this field is required to accommodate more than just a single Canadian call sign of maximum 6 characters, but also Canadian call signs being used in the United States. Such call signs must be suffixed with the call zone in which they are being used. [1]

For example, the Canadian amateur call sign of `VA3INI` would become `VA3INI/W5`, where `W5` is the 5th call zone (coincidentally the call zone for the location where Spaceport America Cup is hosted).

There are 10 call zones in the United States, represented using digits 0-9. [2] The digit is preceded by a forward slash and a capital 'W', for a total of three additional characters on top of the original 6 for a Canadian HAM radio call sign.

Length The length field indicates the total length of the packet in bytes. This includes the packet header itself.

The length of a packet must always be a multiple of four bytes. Because of this the length field is encoded in units of four bytes per least significant bit. Because a packet with a length of zero is not possible, the value in the length field is one less than the length of the packet divided by four. This results in a possible range of packet lengths of 4 to 1024 bytes in multiples of four (though a valid packet must have a length of at least 16 bytes).

The following equations show how the length of a packet in bytes can be converted to the corresponding length value and vice versa.

$$\text{Length} = \frac{\text{packet length in bytes}}{4} - 1$$
$$\text{packet length in bytes} = (\text{Length} + 1) \cdot 4$$

Version The *version* field indicates what version of the radio packet format the packet adheres to. The receiver should not attempt to parse a packet that has a version number which it does not recognize. The *call sign* and *version* fields will remain for all future versions of this packet format, all other fields could change.

Source Address This field indicates the source of the packet. Possible device addresses are listed in table 2.1. 0xF is not a valid source address.

Address	Device
0x00	Ground Station
0x01	Rocket
0x02 through 0xFE	Reserved for future use
0xFF	Multicast

Table 2.1: Device addresses

Packet Number This field contains a count value that should be incremented with every packet transmission from a given device.

2.2 Block Header

Every block starts with a four byte block header. This header indicates the type of information contained in the block as well as the blocks's intended recipient. The block header is formatted as described in Figure 2.3.

0 1 2 3 4 5 6 7	8 9 10 11 12 13 14 15	16 17 18 19 20 21 22 23	24 25 26 27 28 29 30 31
Block Length	Type	Subtype	Dest. Addr.

Figure 2.3: Block header format

Block Length The block length indicates how many bytes are in the block, including the block header. The offset of the block after the current block is equal to the sum of the offset of the current block and the current blocks length.

The length of a block must always be a multiple of four, therefore the block length field is encoded in units of four bytes per least significant bit. Because a valid block will always contain at least 4 bytes, the value of the block Length field is one less than the length of the block divided by four. This means that the field has a range from four bytes to 128 bytes in four byte increments.

The following equations show how the length of a block in bytes can be converted to the corresponding length value and vice versa.

$$\text{Block Length} = \frac{\text{block length in bytes}}{4} - 1$$
$$\text{block length in bytes} = (\text{Block Length} + 1) \cdot 4$$

If the length of a block is such that some or all of the block would exceed the total packet length, then the block is invalid and will be ignored.

Type The block type field indicates the purpose of the block. Possible values are listed in table 2.2. Block types are discussed in more detail in section 3.

Value	Block Type
0x00	Data
0x01 through 0xFF	Reserved for future use

Table 2.2: Block types

Sub-type The sub-type indicates the type of information or request contained in the block. The possible block sub-types for each block type are listed and discussed in more

detail in section 3.

Destination Address This field indicates what device the block is intended for. Received blocks which are intended for a different device should be ignored. Possible addresses are listed in table 2.1. A destination address of 0xFF indicates that the block is intended for anyone who receives it. 0xFF indicates that the block is intended for anyone who receives it.

3 Block Formats

Blocks have a type which indicates their purpose. The possible block types are listed in table 2.2. This section describes the possible block sub-types and their associated data formats.

3.1 Data Blocks

Data blocks contain telemetry data. The possible sub-types data blocks are listed in table 3.1.

Note that all **mission time** fields are measured in milliseconds since launch, and are all unsigned 32-bit integers.

Any integer fields larger than 1 byte are in little-endian format. Any unsigned integers are in two's complement format. This is because fields are packaged using C structs on a little endian machine (the de-facto standard).

Value	Description
0x00	Debug message
0x01	Altitude
0x02	Temperature
0x03	Pressure
0x04	Acceleration
0x05	Angular velocity
0x06	GNSS location
0x07	GNSS metadata
0x08	Humidity
0x09 through 0xFF	Reserved for future use

Table 3.1: Data Block Subtypes

3.1.1 Debug Message

Debug messages are string messages intended to provide human readable debugging output. They are encoded as UTF-8 strings and do not require a terminating NULL character because the string length can be extrapolated from the block length. Every debug message is preceded by a 32 bit unsigned mission time value. See Figure 3.1.

Note that like all other data blocks, debug messages must be a multiple of four bytes long. If the string to be logged is not a multiple of four bytes it must be padded with NULL characters at the end to make it fits the required alignment. The length recorded in the block header must be a multiple of four, and therefore must include any NULL padding bytes at the end of the string.

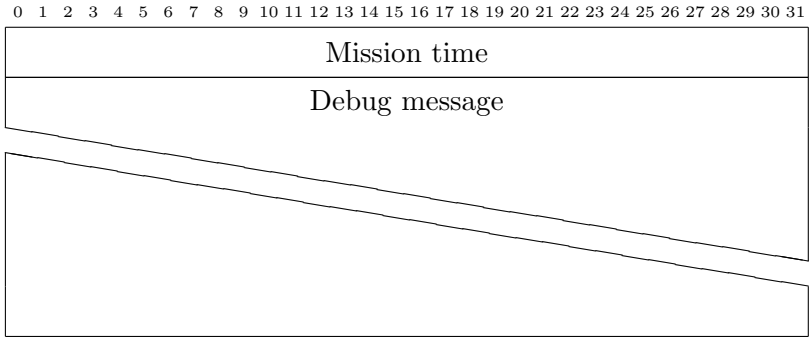

Figure 3.1: Layout of debug message data block

3.1.2 Altitude

The altitude block is used to convey the altitude of the rocket. The format of the altitude block payload is described in figure 3.2.

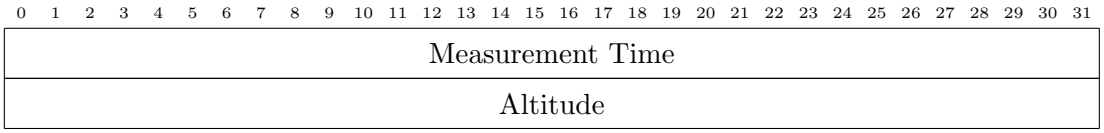

Figure 3.2: Altitude Data Payload Format

Measurement Time The mission time when the measurement was taken.

Altitude The calculated altitude in units of 1 mm/LSB. This field is a signed 32 bit integer in two's complement format.

3.1.3 Temperature

The temperature data block reports the temperature detected from within the rocket in millidegrees Celsius. The format of the temperature block payload is described in figure 3.3.

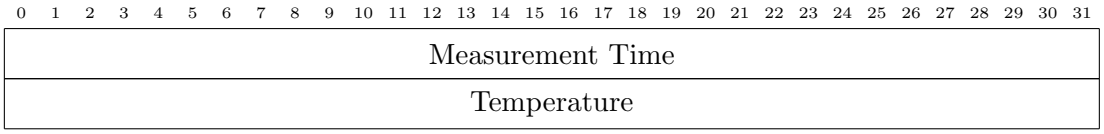

Figure 3.3: Temperature data block format

Measurement Time The mission time when the measurement was taken.

Temperature The measured temperature in units of millidegrees Celsius. This field is a signed 32 bit integer in two's complement format.

3.1.4 Pressure

The pressure data block reports the pressure detected from within the rocket in Pascals. The format of the pressure block payload is described in figure 3.4.

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
Measurement Time
Pressure

Figure 3.4: Pressure data block format

Measurement Time The mission time when the measurement was taken.

Pressure The measured pressure in units of Pascals. This field is an unsigned 32 bit integer.

3.1.5 Acceleration

The acceleration block is used to convey generic 3-axis accelerometer data. This block is intended to abstract over the details of any individual accelerometer. The format of the acceleration block payload is described in figure 3.5.

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31		
Measurement Time		
Full Scale Range		X-Axis
Y-Axis		Z-Axis

Figure 3.5: Acceleration Data Payload Format

Measurement Time The mission time when the measurement was taken.

Full Scale Range The full scale range of the accelerometer in g. This value represents the maximum and minimum acceleration that can be measured, for example a value of 16 in this field represents a full scale range of $\pm 16\,\mathrm{g}$. If the resolution of the accelerometer is less than 16 bits the full scale range must be adjusted to compensate, for example a 10 bit accelerometer with a $\pm 2\,\mathrm{g}$ full scale range would use the value 64 for this field.

This value can be used to calculate the accelerometer sensitivity, which in turn can be used to convert the acceleration measurements for each axis into useful units. For a measurement m with a full scale range f the acceleration in g can be found like this:

$$\text{acceleration} = m \cdot \frac{f}{2^{15}}$$

***-Axis** These fields represent the acceleration measurements for each axis. These fields are signed integers in two's complement format. If the resolution of the accelerometer is less than 16 bits the values must be sign extended.

3.1.6 Angular Velocity

The angular velocity block is used to convey generic 3-axis gyroscope data. This block is intended to abstract over the details of any individual gyroscope. The format of the acceleration block payload is described in figure 3.6.

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15	16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
Measurement Time	
Full Scale Range	X-Axis
Y-Axis	Z-Axis

Figure 3.6: Angular Velocity Data Payload Format

Measurement Time The mission time when the measurement was taken.

Full Scale Range The full scale range of the gyroscope in degrees per second. This value represents the maximum and minimum angular velocity that can be measured, for example a value of 2000 in this field represents a full scale range of $\pm 2000\,\text{degreespersecond}$. If the resolution of the gyroscope is less than 16 bits the full scale range must be adjusted to compensate, for example a 12 bit gyroscope with a $\pm 500\,\text{degreepersecond}$ full scale range would use the value 4000 for this field.

This value can be used to calculate the gyroscope sensitivity, which in turn can be used to convert the acceleration measurements for each axis into useful units. For a measurement m with a full scale range f the angular velocity in degrees per second can be found like this:

$$\text{angular velocity} = m \cdot \frac{f}{2^{15}}$$

***-Axis** These fields represent the angular velocity measurements for each axis. These fields are signed integers in two's complement format.

3.1.7 GNSS Location

The format of the GNSS location block is described in figure 3.7.

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15	16 17 18 19 20 21 22 23	24 25	26 27 28 29 30 31
Fix Time			
Latitude			
Longitude			
UTC Time			
Altitude			
Speed	Course		
PDOP	HDOP		
VDOP	Sats	Fix	

Figure 3.7: GNSS Location Data Payload Format

Fix Time The mission time when the fix was received.

Latitude The latitude of the rocket in units of 100 µ′/LSB (micro-arcminutes per least significant bit). This field is a signed 32 bit integer in two's complement format.

Longitude The longitude of the rocket in units of 100 µ′/LSB (micro-arcminutes per least significant bit). This field is a signed 32 bit integer in two's complement format.

UTC Time The UTC time when the fix was received in seconds since the Unix epoch.

Altitude The altitude as calculated by the GNSS module in units of millimetres above sea level. This field is a signed 32 bit integer in two's complement format.

Speed The speed over ground of the rocket in hundredths of a knot. This field is a signed 16 bit integer in two's complement format.

Course The course over ground of the rocket in hundredths of a degree. This field is a signed 16 bit integer in two's complement format.

PDOP Position Dilution of Precision * 100.

HDOP Horizontal Dilution of Precision * 100.

VDOP Vertical Dilution of Precision * 100.

Sats The number of GNSS satellites used in the fix.

Fix The fix type as shown in table 3.2.

Fix Type Value	Description
0x0	Unknown
0x1	Not available
0x2	2D fix
0x3	3D fix

Table 3.2: GNSS Fix Type Values

3.1.8 GNSS Metadata

The format of the GNSS location block is described in figure 3.8. The length of this block varies, after the GLONASS satellites in use bit field 4 bytes of information is encoded for each satellite in view of the GNSS module. Note that not all satellites which are in view will also be in use.

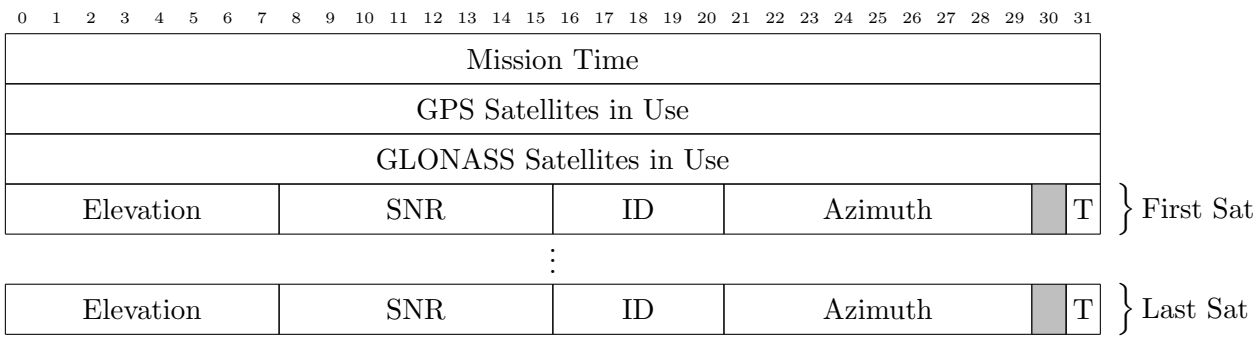

Figure 3.8: GNSS Metadata Payload Format

Mission Time The mission time when the fix was received.

GPS Satellites in Use This bit field indicates which GPS satellites are used in the current fix. Each bit position represents a GPS pseudo-random noise sequence.

GLONASS Satellites in Use This bit field indicates which GLONASS satellites are used in the current fix. Each bit position represents a slot number. Bit zero corresponds to slot 65, bit 1 to slot 66 and so on.

Elevation This field represents the elevation for a GNSS satellite in degrees.

SNR This field represents the signal to noise ratio for a signal from a satellite in dB Hz.

ID This field contains the satellite pseudo-ransom noise sequence for GPS satellites or the satellite ID for GLONASS satellites.

Azimuth This field contains the satellite azimuth in degrees.

T This field encodes the satellite type, it is 0 for GPS satellites or 1 for GLONASS satellites.

3.1.9 Humidity

The humidity block is used to convey the relative humidity inside of the rocket. The format of the humidity block is described in figure 3.9.

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
Measurement Time
Humidity

Figure 3.9: Humidity Data Payload Format

Measurement Time The mission time when the measurement was taken.

Humidity The calculated relative humidity in ten thousandths of a percent (i.e 1% is 100). This field is an unsigned 32 bit integer.

References

[1] S. Bertuzzo. "Operating amateur radio in foreign countries." (), [Online]. Available: https://www.rac.ca/operating/operation-in-foreign-countries/ (visited on 02/11/2024).

[2] S. Stroobandt. "Us call areas & zones by state." (2019), [Online]. Available: https://hamwaves.com/map.us/en/index.html (visited on 02/11/2024).